CS 147 — Verilog Rules

Spring 2026

Adapted from "Use of Verilog in CS/ECE 552" (Spring 2008 version)
Original document adapted from Andy Phelps (CS552-Sp'06)
Last Modified: January 2026

Contents

1. Introduction

In CS 147, you will be limited to a small subset of the complete Verilog language. The purpose of this document is to outline and explain these usage restrictions. It is not a complete Verilog reference.

You can consult the following resources for a more complete documentation of the language:

- On-line Verilog HDL Quick Reference Guide
- ASIC World Verilog Quick Reference

2. Justification of Limitation

The intention of limiting the Verilog language is to force you to "think like hardware". Verilog in its full form is a powerful HDL that can very nearly be used as a programming language with capabilities similar to C.

While this is useful for experienced designers trying to increase productivity, there is no intellectual benefit to be gained by using the abstract features of Verilog. Instead, you will write Verilog in a manner that reflects the actual hardware you are trying to design. Think of it as verbally describing the components of a schematic sheet.

In addition to the educational value of using a subset of the Verilog specification, limiting your code to only well-understood constructs will also help to avoid some pitfalls which can lead to hard to find bugs. Designing with a limited subset of Verilog will tend to produce good designs which synthesize well; for this reason you may be required to follow similar rules when you have a job in industry.

3. Allowed Keywords

Verilog keywords in this course are grouped into three categories: always allowed, allowed with stipulations, and never allowed. This list is not guaranteed to be complete. If you are ever in doubt about the use of a Verilog keyword consult the TA before proceeding.

Verilog Keyword Summary (Table 1)

Always Allowed	Allowed with Stipulations	Never Allowed
assign	case	attribute
module	casex	buf
endmodule	casez	bufif0
input	reg	bufif1
output	always	pulldown
wire	begin	pullup
define	end	rcmos
parameter		real
		cmos
		deassign
		disable
		edge
		else
		endattribute
		endfunction
		endprimitive
		endspecify
		endtable
		endtask
		event
		for
		forever
		fork
		function
		highz0
		highz1
		if
		ifnone
		initial
		inout
		integer
		join
		medium
		large
		macromodule
		negedge
		nmos
		notif0
		notif1
		pmos
		posedge
		primitive
		pull0
		pull1
		realtime
		release
		repeat
		rnmos
		rpmos
		rtran
		rtranif0
		rtranif1
		scalared

Stipulations

Keywords in the allowed with stipulations group can only be used in conjunction with a case statement. The following stipulations apply:

- `case`, `casex` — the `default` clause is required. If the default clause is used during execution, an error must be asserted. Large case statements (∼12 lines or more) must go in their own module.
- `reg` — can only be used to specify the outputs of a case statement.
- `always` — can only be used to introduce a case statement.
- `begin` — can only be used to introduce a case statement.
- `end` — can only be used to terminate a case statement.

4. Allowed Operators

In this course you will be limited to using simple boolean logic operators. You may use the shift operators, but only by a constant amount (i.e. `sigY << sigX` is not allowed, but `sigY << 4` is). The ternary operator (`a ? b : c`) is allowed and even encouraged. Concatenation ({`bit, bit`}) and reduction (`&a[31:0]`) are allowed. Logical equality (==) and inequality (`!=`) operators are also permitted.

You are expressly forbidden from using arithmetic operators (`+`, `-`, `*`, `/`, `%`) and less-than or greater-than comparisons.

Operator Summary (Table 2)

Allowed		Not Allowed	
`~m`	Inversion	`m + n`	Addition
`m & n`	Bitwise AND	`m - n`	Subtraction
`m | n`	Bitwise OR	`m * n`	Multiplication
`m ^n`	Bitwise XOR	`m / n`	Division
`m ~^n`	Bitwise NXOR	`m % n`	Modulus
`&m`	AND Reduction	`-m`	Negation (2's Comp)
`~&m`	NAND Reduction	`m && n`	Logical AND
`|m`	OR Reduction	`m || n`	Logical OR
`~|m`	NOR Reduction	`m < n`	Less Than
`^m`	XOR Reduction	`m > n`	Greater Than
`~^m`	NXOR Reduction	`m <= n`	Less Than or Equals
`m == n`	Equality	`m >= n`	Greater Than or Equals
`m != n`	Inequality		
`m === n`	Identity		
`m !== n`	Not Identical		
`m << const`	Shift Left*		
`m >> const`	Shift Right*		
`test ? m:n`	Ternary		
{`m, n`}	Concatenation		
{`m`{`n`}}	Replication		

5. Usage Examples

The most fundamental thing to keep in mind is that your hardware design will be composed of two things: combinational logic, and state. For any moderately complex logic design, state must be handled in an organized way. An R-S flipflop would get you laughed out the door of a computer design company! In our designs, state will all be held in D-flipflops that are all clocked on the rising edge of a common system clock.

You are provided (in the course infrastructure) with a module for a single D-flipflop with synchronous reset. Instantiate copies of this module to build larger registers. Do not create your own flipflops in some other way.

Combinational logic will be specified primarily with `assign` statements. The statement `assign xyz = a & b;` specifies an AND gate. Assign statements can get long and complex, and can specify hundreds of gates, but it always represents some set of flow-through logic that generates the value for a wire (or bundle of wires).

Your design will be hierarchical. That means you will write Verilog modules for all the small

building blocks of your design, and you will instantiate these modules within larger modules.

The modules you write should have a structure like this:

```
module my_module_name(param,param,...param);
  input param;
  output param;
  wire [msb:lsb] signalName;
  assign signalName = expression;
  modulename instance(param,param...); // call some submodule
endmodule
```

Note that assign statements can operate on an entire vector of bits at one time. For example:

```
wire [15:0] a,b,ss,ans;
assign ss = {16{s}}; // use 16 copies of "s"
assign ans = (ss ^ a) | b;
```

When instantiating a module, you must name the ports when connecting wires to it. For example:

```
mux2_1 m0(d0,d1,s,b) // is NOT OK
mux2_1 m0(.input0(d0),.input1(d1),.select(s),.output0(b)) // is CORRECT
```

Case statements are the only situation in this class where you will use statements such as `begin`, `end`, `reg`, `always`. Example:

```
wire [1:0] s;
reg out;
always @* case(s)
  2'b00: out=i0;
  2'b01: out=i1;
  2'b10: out=i2;
  2'b11: out=i3;
endcase
```

Here is a larger example:

```
wire inputA,inputB,goBack;
wire [2:0] currentState;
reg [2:0] newState;
reg out,err;

always @(goBack or currentState or inputA or inputB)
casex({goBack,currentState,inputA,inputB})
  6'b1_???_?_?: begin out=0; newState=3'b000; err=0; end
  6'b0_000_0_?: begin out=0; newState=3'b000; err=0; end
  6'b0_000_1_?: begin out=0; newState=3'b001; err=0; end
  6'b0_001_1_?: begin out=0; newState=3'b001; err=0; end
  6'b0_001_0_0: begin out=0; newState=3'b010; err=0; end
  6'b0_001_0_1: begin out=0; newState=3'b011; err=0; end
```

```
  6'b0_010_?_0: begin out=0; newState=3'b010; err=0; end
  6'b0_010_?_1: begin out=0; newState=3'b011; err=0; end
  6'b0_011_?_1: begin out=0; newState=3'b011; err=0; end
  6'b0_011_?_0: begin out=0; newState=3'b100; err=0; end
  6'b0_100_?_?: begin out=1; newState=3'b000; err=0; end
  6'b0_101_?_?: begin out=0; newState=3'b000; err=1; end
  6'b0_110_?_?: begin out=0; newState=3'b000; err=1; end
  6'b0_111_?_?: begin out=0; newState=3'b000; err=1; end
  default: begin out=0; newState=3'b000; err=1; end
endcase
```

Also not allowed: any notion of time or delays; any real numbers. For statements are OK for debugging but must not appear in a finished assignment.

6. CS 147 Verilog Check Program

A Java program `Vcheck` will be used to scan your design for some of the common illegal constructs. You will be required to run it on your Verilog designs and hand in the output.

- Method 1: copy `Vcheck.class` and `VerFile.class` into a directory, then run:

  ```
  java Vcheck < myfile.v
  ```

- Method 2: from a course machine shell prompt, run:

  ```
  vcheck.sh < myfile.v
  ```

- Method 3: run on all Verilog files in a directory:

  ```
  vcheck-all.sh
  ```